Project Type: Engineering Project
Document Type: Architecture & Design Specification

Project Type: Final-Year Engineering Project (Cybersecurity + AI + Data Engineering) **Document Type:** Architecture & Design Specification **Primary Language:** Python (backend, AI/ML, data pipelines), TypeScript/JavaScript (frontend, browser extension) **Document Status:** Implementation-Ready Draft v1.0 **Team Size:** 6

Document Purpose

This document is the single source of architectural truth for the Cyber Threat Intelligence Hub (CTI Hub). It exists so that six people working on different layers of the same system — frontend, data engineering, AI, databases, detection research, and design — can build independently without their work colliding at integration time. Every interface, schema, and technology choice that appears here is a contract between team members, not a suggestion.

The document deliberately compares 2–3 candidate technologies per major decision before converging on one final stack (Section 20), because a final-year team benefits from seeing *why* an option was rejected, not just what was picked. If a team member later wants to swap a component, the trade-off table tells them what they're giving up.

Table of Contents

1. Project Vision
2. Functional Requirements
3. Non-Functional Requirements
4. Recommended Technology Stack (Comparative)
5. Technology Compatibility Matrix
6. Overall System Architecture
7. Microservice Breakdown
8. Data Flow
9. Database Design
10. AI Architecture
11. Threat Intelligence Pipeline
12. External Integrations
13. Security Architecture
14. Deployment Architecture
15. Folder Structure
16. API Design
17. Database ER Diagram
18. Sequence Diagrams
19. Compatibility & Integration Report
20. Final Recommended Stack

---

1. Project Vision

1.1 Goals

- Build a unified platform that pulls cyber threat intelligence (CVEs, malware indicators, OSINT, threat-actor data) from multiple external sources into one normalized data layer.
- Apply AI to reduce analyst workload: summarizing long advisories, scoring risk, and classifying threats automatically instead of requiring manual triage.

- Give security teams — or, in this project's case, evaluators and recruiters — a working demonstration of how modern AI engineering, data engineering, and cybersecurity practice fit together in one coherent system.
- Ship something that runs comfortably on a student budget (free-tier APIs, open-source components, a single cloud VM or even a laptop) but is architected the way a production system would be, so the project scales conceptually beyond the demo.

1.2 Objectives

#	Objective	Measurable Outcome
1	Ingest live CVE and malware feeds	At least 3 external feeds polled on a schedule, data visible in the DB within minutes of publication
2	Normalize heterogeneous threat data	Single unified `threat` schema regardless of source format (JSON, STIX, CSV, plaintext OSINT)
3	AI-generated threat summaries	Every ingested CVE/advisory gets a plain-English summary within seconds of ingestion
4	Automated risk scoring	Each threat gets a 0–100 risk score derived from CVSS, exploit availability, and asset relevance
5	Searchable IOC database	Sub-second search across IPs, hashes, domains via Elasticsearch/OpenSearch
6	Threat actor & campaign tracking	MITRE ATT&CK technique mapping visible per actor/campaign
7	Real-time-ish ingestion	New feed data reflected in the dashboard within a defined SLA (e.g., 5 minutes)
8	Browser-based access everywhere	Full functionality in any modern browser, plus a Chrome extension for in-context lookups
9	Modular, swappable components	Any single service (AI, search, feed collector) can be redeployed independently

1.3 Target Users

- **SOC analysts / security researchers** who need a faster way to triage new CVEs and IOCs without reading every advisory in full.
- **Students and recruiters/evaluators** assessing the project — this is the primary real-world audience for a final-year build, so demo clarity and resume-relevant technology choices matter architecturally, not just cosmetically.
- **Threat hunters** who want quick IOC lookups while browsing other security sites (this is the direct use case for the Chrome extension).
- **Engineering managers / admins** who manage users, feed sources, and system health via an admin panel.

1.4 Business Problem

Security teams today pull threat data from dozens of disconnected sources — NVD, vendor advisories, OSINT feeds, Twitter/X, vendor blogs — and manually decide what matters. This is slow, inconsistent, and doesn't scale with the volume of CVEs published daily (NVD alone publishes thousands per year, with spikes during major disclosure events). Most of that data is unstructured text that a human has to read and interpret before they can act. The CTI Hub addresses this by centralizing ingestion, normalizing the data into one schema, and using AI to do the first pass of summarization and prioritization — so a human only spends time on what the system has already flagged as relevant and urgent.

1.5 Expected Outcomes

- A live, browsable threat intelligence dashboard showing current CVEs, malware campaigns, and IOCs with AI-generated summaries and risk scores.
- A Chrome extension that lets a user highlight an IP, domain, or hash on any webpage and get an instant threat-intel lookup without leaving the page.

- A documented, modular codebase that demonstrates full-stack development, data engineering (ETL, pipelines, schema design), AI/ML engineering (NLP summarization, risk modeling), and security research (MITRE mapping, IOC analysis) — giving all six team members a substantial, individually attributable contribution.
- A system architecture that could plausibly be pitched to a real organization as a starting point for an internal threat intel platform, not just a class project.

---

2. Functional Requirements

Functional requirements are grouped by capability area and mapped to the team member(s) primarily responsible, so ownership is unambiguous from the start.

2.1 Threat Feed Collection & Ingestion

- FR-1: Poll NVD/CVE API on a configurable schedule and ingest new/updated CVEs.
- FR-2: Poll CISA KEV (Known Exploited Vulnerabilities) feed and flag matching CVEs as "actively exploited."
- FR-3: Pull malware/IOC intelligence from AlienVault OTX.
- FR-4: Pull reputation data from AbuseIPDB and VirusTotal for IPs/hashes/domains seen in ingested threats.
- FR-5: Support pluggable feed connectors so a new source can be added without touching core ingestion logic.
- FR-6: Deduplicate records across feeds (e.g., same CVE referenced by two sources).
- FR-7: Track per-feed ingestion health (last successful pull, error count, latency).

2.2 Data Normalization & Storage

- FR-8: Normalize all ingested records into a single canonical `Threat` schema regardless of source format.
- FR-9: Extract IOCs (IPs, domains, hashes, URLs, file names) from free-text advisories via pattern matching / NLP.
- FR-10: Store structured threat data in a relational database; store full-text/advisory content in a document store or search index.
- FR-11: Version threat records so updates to a CVE (e.g., CVSS score changes) are tracked, not overwritten silently.

2.3 AI / NLP Capabilities

- FR-12: Generate a plain-English summary (2–4 sentences) for every ingested CVE/advisory using an LLM.
- FR-13: Generate a risk score (0–100) per threat using a model that combines CVSS base score, exploit-availability signals (KEV presence), and asset/keyword relevance.
- FR-14: Classify threats by category (e.g., RCE, privilege escalation, DoS, info disclosure) using zero-shot or fine-tuned classification.
- FR-15: (Optional) Provide a conversational chatbot that answers natural-language questions against the threat database ("what critical CVEs affect Apache this week?").
- FR-16: Log every AI inference (prompt, model, response, latency, cost) for evaluation and debugging.

2.4 Threat Actor & Campaign Tracking

- FR-17: Maintain a database of known threat actors/groups with associated TTPs.
- FR-18: Map ingested threats and IOCs to MITRE ATT&CK techniques where possible.
- FR-19: Group related IOCs/threats into "campaigns" (manually curated and/or AI-assisted clustering).

2.5 Search & Discovery

- FR-20: Full-text search across CVEs, advisories, and threat summaries.
- FR-21: Structured IOC search (exact match and fuzzy match) for IPs, hashes, domains, URLs.
- FR-22: Filter/search by severity, date range, vendor/product, actor, campaign, ATT&CK technique.
- FR-23: Saved searches / search alerts (optional stretch goal).

2.6 Dashboard & Visualization

- FR-24: Overview dashboard showing recent critical CVEs, trending IOCs, and active campaigns.
- FR-25: Severity/risk distribution charts (e.g., CVEs by CVSS band over time).
- FR-26: Threat actor / campaign visualization (timeline or graph view).
- FR-27: Drill-down detail view per threat: summary, raw advisory, IOCs, risk score breakdown, ATT&CK mapping, related threats.

2.7 User Management & Authentication

- FR-28: User registration/login with email + password, plus OAuth (Google/GitHub) as an option.
- FR-29: Role-based access control: Admin, Analyst, Viewer (at minimum).
- FR-30: Session management via JWT with refresh tokens.
- FR-31: Per-user saved preferences (dashboard layout, default filters, watchlists).

2.8 Admin Panel

- FR-32: Manage feed sources (enable/disable, adjust polling frequency, view health).
- FR-33: Manage users and roles.
- FR-34: View system-wide metrics (ingestion volume, AI cost/usage, API latency).
- FR-35: Trigger manual re-ingestion or reprocessing of a feed.

2.9 Browser Extension

- FR-36: Highlight-and-lookup: select an IP/domain/hash/CVE ID on any webpage, right-click or use a popup to query the Hub's API.
- FR-37: Extension popup shows a condensed threat summary, risk score, and a link to the full dashboard entry.
- FR-38: Extension authenticates against the same backend using the user's existing account (shared JWT or extension-specific token).
- FR-39: Extension works offline-gracefully (clear "no connection" state rather than silent failure).

2.10 Notifications (Optional/Stretch)

- FR-40: Email or in-app notification when a watchlisted vendor/product has a new critical CVE.
- FR-41: Webhook support for integrating with external tools (Slack, Teams).

2.11 Detection Engineering Support

- FR-42: Auto-suggest or link relevant Sigma/YARA rules for a given threat where available (manually curated initially).
- FR-43: Export IOC sets in common formats (CSV, STIX 2.1 bundle) for use in other security tooling.

---

3. Non-Functional Requirements

Category	Requirement	Target / Notes
Scalability	System must handle growth in feed volume and user load without redesign	Stateless API services behind a load balancer; horizontally scalable workers for ingestion
Performance	Dashboard and search must feel responsive	API p95 latency < 500ms for read endpoints; search queries < 300ms
Availability	Core read paths (dashboard, search) should tolerate individual component failure	Target 99% uptime for the demo deployment; AI service failure must degrade gracefully (show threat without summary rather than erroring)
Security	Platform handling threat data must itself be secure	TLS everywhere, JWT auth, RBAC, input validation, secrets in a vault/manager — detailed in Section 13

Category	Requirement	Target / Notes
Reliability	Feed ingestion must not silently fail	Retry with backoff, dead-letter handling for failed records, alerting on repeated feed failures
Maintainability	Codebase must be navigable by 6 different contributors	Consistent project structure (Section 15), typed Python (Pydantic/type hints), linting/formatting enforced in CI
Modularity	Services should be independently deployable	Microservice-oriented design (Section 7) with clear API boundaries; feed connectors as plugins
Portability	Should not be locked to one cloud provider	Containerized (Docker) everywhere; cloud-managed services chosen with open-source equivalents available
Logging	All services must produce structured, correlatable logs	JSON logs with request/correlation IDs; centralized log aggregation
Monitoring	System health must be observable	Metrics (latency, error rate, ingestion throughput) exposed via Prometheus-compatible endpoints; dashboards in Grafana
Testability	Core logic must be unit-testable independent of external APIs	External feed clients mocked in tests; pytest coverage tracked in CI
Data Integrity	Threat data changes must be auditable	Append-only history table for threat record changes; no hard deletes on core entities
Cost-effectiveness	Must run within a student/free-tier budget	Open-source-first stack; LLM calls batched/cached to control API cost

4. Recommended Technology Stack (Comparative)

Each category below presents 2–3 realistic candidates, weighed against this project's actual constraints: a 6-person student team, Python as the preferred core language, a browser-first web app, and a Chrome extension. Section 20 converges these into one final stack.

4.1 Frontend

Option	Pros	Cons	Verdict
React + TypeScript	Huge ecosystem, component libraries (shadcn/ui, Recharts) map directly to dashboard needs, same mental model reusable in the Chrome extension's popup UI, most resume-relevant	Not Python — but frontend was never going to be Python regardless of backend choice	**Selected**
Vue 3 + TypeScript	Gentler learning curve, good docs, Composition API is clean	Smaller ecosystem for security-dashboard-style components than React; less common in job postings than React	Rejected — no architectural advantage over React for this project's needs
Server-rendered Jinja2 templates (Python-only, via FastAPI)	Truly 100% Python, zero JS build tooling, fastest to start	Cannot reasonably power a rich interactive dashboard (live charts, filters, drag interactions) or share component logic with a Chrome extension popup; would need a second UI stack for the extension anyway	Rejected — fails FR-24–27 and the extension requirement outright

Reasoning: The instruction to keep Python as the preferred language applies most naturally to backend, data engineering, and AI — not to the browser UI, which is inherently a JS/TS/HTML/CSS environment regardless of backend choice. Using React for both the main dashboard and the Chrome extension popup means one component language for both browser surfaces, which directly reduces total surface area the team has to maintain.

4.2 Backend / API Framework

Option	Pros	Cons	Verdict
FastAPI (Python)	Async-native, automatic OpenAPI docs, Pydantic validation built in, excellent for AI/ML integration since it shares the Python runtime with the AI layer, fast to develop	Younger ecosystem than Django for things like admin panels (mitigated — we build a custom admin UI anyway)	**Selected**
Django + DRF	Batteries-included (admin panel, ORM, auth)	Heavier, more opinionated; async support is improved but still secondary to FastAPI's; less natural fit for an AI-inference-heavy service	Rejected — admin panel and ORM benefits don't outweigh async/AI fit
Node.js (Express/Nest)	Single language with frontend (JS/TS everywhere)	Breaks the Python-first requirement for backend; would split AI/data work (Python) from API work (JS) across a hard language boundary, which is worse for a small team than a JS/TS UI + Python backend split	Rejected — explicitly against stated language preference

Reasoning: FastAPI keeps backend, data engineering, and AI work in one language and one runtime family, which matters a lot for a 6-person team where the same person may touch ETL and API code in the same week.

4.3 Database (Primary / Relational)

Option	Pros	Cons	Verdict
PostgreSQL	Mature, strong JSON support (JSONB) for semi-structured threat fields, excellent Python driver support (SQLAlchemy/asyncpg), free, widely used in industry	None significant for this use case	**Selected**
MySQL/MariaDB	Familiar, widely hosted	Weaker JSON/array support than Postgres, fewer advanced indexing options useful for threat data (GIN indexes, full-text)	Rejected — Postgres is strictly better for this data shape
SQLite	Zero setup, fine for early prototyping	Cannot support concurrent multi-service writes at any real scale; wrong choice the moment ingestion workers and the API both write	Rejected for production data layer (acceptable only for local unit tests)

4.4 Search / IOC & Full-Text Index

Option	Pros	Cons	Verdict
OpenSearch	Open-source (Apache 2.0) fork of Elasticsearch, same query API/DSL, no licensing ambiguity, strong full-text +	Slightly behind Elasticsearch in newest features	**Selected**

Option	Pros	Cons	Verdict
	structured filtering, good for IOC search and dashboards (Kibana-equivalent: OpenSearch Dashboards)		
Elasticsearch	Best-in-class search, huge community knowledge base	Licensing (SSPL/Elastic License) is more restrictive for redistribution; OpenSearch covers this project's needs with a cleaner license	Rejected — license friction with no functional gain here
PostgreSQL full-text search only (no separate search engine)	One less moving part	Noticeably weaker for fuzzy IOC matching, relevance ranking, and the search-heavy dashboard filtering in FR-20–23	Rejected — under-powers core search requirements, though kept as a fallback for very small deployments

4.5 Caching / Queue Broker

Option	Pros	Cons	Verdict
Redis	Sub-millisecond cache reads, doubles as a message broker for Celery, pub/sub for real-time dashboard updates, trivial to run in Docker	None significant	**Selected**
Memcached	Simpler, slightly faster for pure key-value caching	No pub/sub, no native data structures, can't double as a task queue broker — would need Redis anyway for FR-7/FR-37 real-time pieces	Rejected — Redis covers caching and removes a second dependency

4.6 Message Queue / Background Jobs

Option	Pros	Cons	Verdict
Celery + Redis broker	Mature Python-native task queue, scheduled tasks (Celery Beat) map directly onto feed-polling needs (FR-1–4), large community, integrates cleanly with FastAPI	Configuration can sprawl if not kept disciplined	**Selected**
RabbitMQ + Celery	More robust message guarantees than Redis-as-broker for very high-throughput systems	Adds a second infrastructure component for a guarantee level this project doesn't need at student scale	Rejected for v1 — documented as the natural upgrade path if ingestion volume grows
Apache Kafka	Industry standard for high-throughput streaming, strong resume value	Heavyweight operationally for a 6-person team's first deployment; massive overkill for "near real-time" at the volume NVD/OTX actually produce	Rejected for v1 — noted as a future-scale option in Section 20

4.7 AI Framework (Orchestration / LLM Access)

Option	Pros	Cons	Verdict
Direct Anthropic/OpenAI SDK calls + LangChain/LlamaIndex only where genuinely needed (RAG)	Full control over prompts and cost, avoids over-abstracting a workload that's mostly "summarize this text" and "classify this text", easiest to evaluate/debug (FR-16)	Team must hand-roll a bit more orchestration glue than a framework would provide	**Selected**
Heavy LangChain-everywhere approach	Pre-built chains for summarization, retrieval, agents	Adds an abstraction layer that often obscures prompt/response debugging — directly works against FR-16 (logging every inference) and against keeping a final-year team's mental model simple	Rejected as the default; LangChain still used narrowly for the optional RAG chatbot (FR-15) where its retrieval chain genuinely saves work
Fully custom in-house orchestration (no libraries at all)	Maximum control	Reinvents retrieval/chunking utilities that LlamaIndex already solves well for the optional chatbot	Rejected — not worth avoiding a well-scoped library for one feature

4.8 ML / NLP Framework (Risk Scoring & Classification)

Option	Pros	Cons	Verdict
scikit-learn (risk scoring) + Hugging Face Transformers (zero-shot classification)	scikit-learn is the right weight for a tabular risk-scoring model (CVSS + KEV + keyword features → 0–100 score); HF Transformers gives a pretrained zero-shot classifier (e.g., `facebook/bart-large-mnli`) for threat categorization without training data the team doesn't have	Zero-shot is less accurate than a fine-tuned model	**Selected**
Full custom deep-learning model (PyTorch from scratch)	Maximum customization, strongest ML resume signal if done well	Requires labeled training data the team doesn't have yet and time the team doesn't have either; risks becoming the project's biggest risk item for the smallest functional gain	Rejected for v1 — documented as a stretch goal once enough labeled threat data accumulates from real usage
Rules-based scoring only (no ML at all)	Simplest, fully deterministic, easy to explain in a viva/demo	Doesn't satisfy "AI-powered" positioning of the project and is a weaker engineering story	Rejected as the sole approach — used as the deterministic *baseline* inside the hybrid scorer (CVSS/KEV signals are themselves rule-based; ML/LLM contributes the keyword-relevance and classification layer)

4.9 Vector Database (for RAG / chatbot, optional feature)

Option	Pros	Cons	Verdict
pgvector (Postgres extension)	No new database to operate — embeddings live next to the relational threat data already in Postgres, simplest ops story for a student team	Less feature-rich than dedicated vector DBs at very large scale	**Selected**
Qdrant / Weaviate (standalone vector DB)	Purpose-built, faster at large scale, more advanced filtering	One more service to deploy, monitor, and secure for a feature (FR-15) that's explicitly optional	Rejected for v1 — natural upgrade path noted if the chatbot becomes a core feature with heavy usage
Chroma (embedded)	Extremely easy local prototyping	Not built for multi-service production access the way pgvector or Qdrant are	Rejected beyond local prototyping

4.10 Authentication

Option	Pros	Cons	Verdict
Custom JWT auth (FastAPI + `python-jose`/`passlib`) with optional OAuth (Google/GitHub) via Authlib	Full control, no vendor lock-in, works identically for the web app and the Chrome extension (just send the JWT), free	More code to write and secure correctly than a managed service	**Selected**
Auth0 / Clerk (managed auth)	Fast to set up, handles a lot of security details for you	Recurring cost beyond free tier, less educational value for a project explicitly meant to demonstrate security engineering, adds an external dependency for a core security function	Rejected — a CTI platform's own auth is exactly the kind of thing this project should build and document well, not outsource
Firebase Auth	Free tier generous, quick	Same "outsources the security story" issue as Auth0; also nudges the whole stack toward Google Cloud lock-in	Rejected

4.11 Cloud Platform

Option	Pros	Cons	Verdict
Any major cloud (AWS/GCP/Azure free tier) running Dockerized services, OR a single VPS (e.g., a $5–10/mo box)	Portable — same Docker Compose setup works on a laptop, a VPS, or a cloud VM; no architectural lock-in	Requires the team to manage the VM/orchestration themselves rather than using fully-managed PaaS	**Selected** (cloud-agnostic, containers-first)
Fully managed PaaS (Heroku, Render, Railway)	Very fast to deploy, less ops burden	Less representative of how production cybersecurity infrastructure is actually run; free tiers are often too small for a multi-service stack (DB + search + cache + queue + AI service)	Rejected as primary path — acceptable for hosting just the frontend/demo if needed, not the full stack
Self-hosted on-prem only	No cloud cost at all	Not accessible to evaluators/recruiters without VPN	Rejected

Option	Pros	Cons	Verdict
		access; defeats "browser-accessible to anyone" goal	

4.12 Containerization & Orchestration

Option	Pros	Cons	Verdict
Docker + Docker Compose	Exactly matches this project's scale (one team, one deployment target, ~8–10 services), trivial local dev parity, easy to hand off	Doesn't auto-scale across multiple machines	**Selected for v1**
Kubernetes	Industry-standard at real scale, strong resume value	Significant operational overhead for a single-deployment student project; the orchestration complexity Kubernetes solves (multi-node scheduling, rolling updates across a fleet) doesn't exist yet at this project's scale	Documented as the Section 14 "optional" production-scale path, not required for v1

4.13 CI/CD

Option	Pros	Cons	Verdict
GitHub Actions	Free for public/student repos, integrates directly with GitHub (where the team's code already lives), simple YAML pipelines for test+lint+build+push-image	None significant	**Selected**
GitLab CI / Jenkins	Equally capable	Adds a second platform (GitLab) or self-hosted ops burden (Jenkins) with no benefit over GitHub Actions for a GitHub-hosted repo	Rejected

4.14 Monitoring & Logging

Option	Pros	Cons	Verdict
Prometheus + Grafana (metrics), structured JSON logs shipped to a simple aggregator (Grafana Loki)	Open-source, Docker-Compose-friendly, the de facto standard stack so it's a strong learning/resume choice, integrates with FastAPI via `prometheus-fastapi-instrumentator`	Another set of containers to run	**Selected**
Cloud-native monitoring (CloudWatch/Stackdriver)	Zero setup if already on that cloud	Locks the monitoring story to one cloud provider, contradicting the portability NFR	Rejected as primary; can coexist if deployed on that specific cloud
No structured monitoring (just print/console logs)	Zero effort	Fails the Logging/Monitoring NFRs outright and makes debugging ingestion failures (FR-7) much harder	Rejected

4.15 File / Object Storage

Option	Pros	Cons	Verdict
S3-compatible object storage (MinIO self-hosted, or actual S3 if on AWS)	Stores raw advisory dumps, exported STIX/CSV files (FR-43), and any large artifacts outside the DB; MinIO is fully open-source and Docker-deployable, API-compatible with AWS S3 so the code never has to change if you move providers	One more container to run locally (MinIO)	**Selected**
Store everything as DB blobs	Simplest	Bloats Postgres with large binary/text blobs, hurts backup and query performance	Rejected

4.16 Visualization Libraries

Option	Pros	Cons	Verdict
Recharts (dashboards) + react-force-graph or Cytoscape.js (actor/campaign graph views)	Recharts is the standard React charting choice with low setup cost; Cytoscape.js is purpose-built for network/graph visualizations like ATT&CK technique maps and actor-campaign relationships	None significant for this scope	**Selected**
D3.js (hand-rolled everything)	Maximum flexibility	Significant extra development time for charts that Recharts already covers well; better reserved for genuinely custom visuals if time allows	Used selectively, not as the default

4.17 Testing

Option	Pros	Cons	Verdict
pytest (backend/AI/data) + Vitest/React Testing Library (frontend)	Matches each layer's native language, both have first-class FastAPI/React support, both run cleanly in GitHub Actions	None significant	**Selected**

4.18 Documentation

Option	Pros	Cons	Verdict
MkDocs Material (architecture/team docs) + FastAPI's auto-generated OpenAPI/Swagger (API reference)	Markdown-native (matches this very document), free hosting via GitHub Pages, OpenAPI docs stay in sync with code automatically since FastAPI generates them from the route definitions	None significant	**Selected**

4.19 Chrome Extension Stack

Option	Pros	Cons	Verdict
Manifest V3 Chrome Extension, React popup (shared component library with the main dashboard), `chrome.storage` for the JWT, content script for text-selection lookups	Manifest V3 is the current required standard (V2 is being deprecated by Google), reusing React components from the main app cuts duplicate UI work, content-	MV3's service-worker model (vs. persistent background pages) requires slightly different state-handling patterns than older tutorials describe	**Selected**

Option	Pros	Cons	Verdict
	script selection lookup directly satisfies FR-36		
Firefox/cross-browser WebExtension from day one	Wider reach	Adds cross-browser testing burden before the core product/AI/data engineering is even proven out; project brief specifically asks for a Chrome extension	Rejected for v1 — WebExtensions API is largely compatible, so cross-browser support is a low-cost future step, not a blocker now

5. Technology Compatibility Matrix

This matrix covers the technologies **selected** in Section 4 (not the rejected alternatives), showing how each integrates with the others it touches directly.

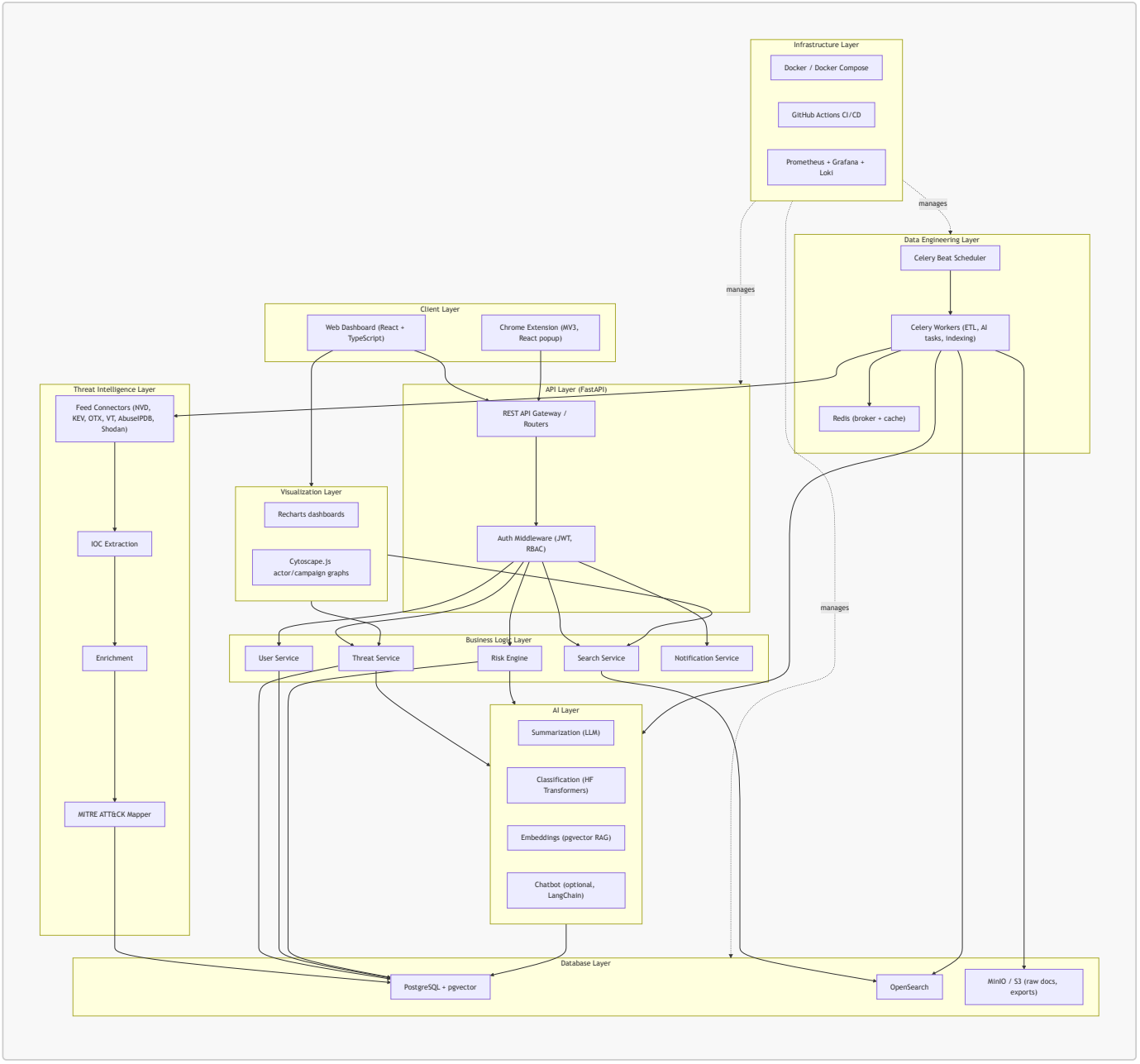
Technology	Compatible With	Reason	Integration Method	Potential Issues
React + TypeScript	FastAPI, Chrome Extension (MV3)	Consumes REST/JSON over HTTP; same component patterns reused in extension popup	`fetch`/`axios` calls to FastAPI's OpenAPI-documented REST endpoints; shared `npm` component package between web app and extension	CORS must be explicitly configured on FastAPI for the web origin and the extension origin (`chrome-extension://`)
FastAPI	PostgreSQL, Redis, OpenSearch, Celery, S3/MinIO, Anthropic/OpenAI SDKs	Python-native async drivers exist for all of these (`asyncpg`/SQLAlchemy async, `redis-py`, `opensearch-py`, Celery client, `boto3`/MinIO SDK)	Direct SDK/driver calls from FastAPI route handlers and dependency-injected service classes	Must manage connection pooling carefully under async load; blocking calls (some embedding/inference SDKs) need to run in a thread pool, not the event loop directly
PostgreSQL	FastAPI (via SQLAlchemy/asyncpg), pgvector extension	Native extension support means vector search lives in the same database/transaction as relational threat data	SQLAlchemy ORM models + Alembic migrations	pgvector requires the extension to be enabled in the Postgres image (`pgvector/pgvector` Docker image) — must not use the default `postgres` image
OpenSearch	FastAPI (via `opensearch-py`), Celery workers (indexing on write)	Threat records written to Postgres are mirrored into OpenSearch asynchronously for search	Dual-write pattern: Postgres is source of truth, OpenSearch indexed via a Celery task triggered on create/update	Risk of index drift if the indexing task fails silently — must alert on indexing failures (ties to Logging/Monitoring NFR)
Redis	FastAPI (cache + pub/sub), Celery	Redis serves two roles at once: API response	`redis-py` client; Celery configured with `redis://`	Must use separate Redis logical DBs (or

Technology	Compatible With	Reason	Integration Method	Potential Issues
	(broker + result backend)	caching and the Celery message broker	broker URL	key prefixes) for caching vs. Celery to avoid key collisions
Celery	Redis (broker), FastAPI (triggers tasks), external feed APIs (NVD, OTX, etc.)	Scheduled polling (Celery Beat) and async background jobs (AI inference, indexing) both run here	FastAPI enqueues tasks via Celery's `.delay()`/`.apply_async()`; Celery Beat schedule defined in config	Long-running AI inference tasks should have explicit timeouts so a stuck external API call doesn't block a worker indefinitely
Hugging Face Transformers / scikit-learn	FastAPI (inline or via Celery worker), Risk Engine service	Pure Python libraries, run in the same runtime as the rest of the backend	Loaded once at service startup (model caching), invoked synchronously inside a Celery task to avoid blocking the API	Transformer models need adequate RAM/CPU (or GPU) — must be containerized in their own service if resource needs diverge from the lightweight API container
Anthropic/OpenAI SDKs	FastAPI (Celery task layer), LangChain/LlamaIndex (RAG chatbot only)	Standard HTTP-based SDKs, async-compatible	Called from within Celery tasks (not the request/response cycle) so summarization never blocks a user-facing API call	Rate limits/cost — must implement caching (don't re-summarize unchanged content) and backoff
pgvector	PostgreSQL, LlamaIndex/LangChain (chatbot RAG layer)	Embeddings stored as a native Postgres column type	`pgvector` SQLAlchemy type + cosine-similarity queries	Embedding dimension must match the chosen embedding model exactly; changing embedding models later requires a backfill migration
Docker / Docker Compose	Every service above	Every component in this stack ships an official or community Docker image	One `docker-compose.yml` defining all services, networks, and volumes	Image version pinning is essential — floating `latest` tags across 8+ services is the most common source of "works on my machine" bugs for student teams
GitHub Actions	Docker, pytest, Vitest	Workflow builds images, runs both test suites, pushes to a container registry	`.github/workflows/ci.yml` with matrix jobs for backend/frontend	Secrets (DB passwords, API keys) must be stored in GitHub Actions secrets, never committed

Technology	Compatible With	Reason	Integration Method	Potential Issues
Prometheus + Grafana	FastAPI, Celery (via exporters)	FastAPI exposes a `/metrics` endpoint; Celery has a Prometheus exporter	Prometheus scrapes `/metrics`; Grafana dashboards query Prometheus	Must avoid high-cardinality labels (e.g., raw IOC values as metric labels) which can overwhelm Prometheus storage
Chrome Extension (MV3)	FastAPI (REST API), React (shared components)	Same JSON contract as the web app; React components bundled via the extension's build step	`chrome.storage.local` for token persistence; content script + background service worker calling the same REST endpoints	MV3 service workers are non-persistent — token refresh logic must be re-entrant rather than relying on long-lived in-memory state

6. Overall System Architecture

The system follows a layered architecture. Each layer has a single clear responsibility and only talks to the layer immediately above or below it (no layer-skipping), which is what keeps six people able to work in parallel without constantly blocking on each other.



Layer responsibilities:

- **Client Layer** — Everything the user directly sees: the React dashboard (browser) and the Chrome extension popup/content script. Talks only to the API Layer over REST.
- **API Layer** — FastAPI routers and auth middleware. Validates requests (Pydantic), enforces JWT/RBAC, and delegates to Business Logic. Contains no business logic itself.
- **Business Logic Layer** — Service classes (Threat, User, Search, Risk Engine, Notification) that implement actual rules: what counts as "critical," how a saved search is evaluated, how a risk score is assembled from its components.
- **AI Layer** — Summarization, classification, embeddings, and the optional chatbot. Invoked by Business Logic and by background workers, never called directly by the Client Layer.
- **Threat Intelligence Layer** — Feed connectors, IOC extraction, enrichment, and ATT&CK mapping. This is where raw external data becomes structured threat-intel domain objects.
- **Data Engineering Layer** — Celery Beat (scheduling), Celery Workers (execution), and Redis (broker/cache) that actually run the ETL and AI/indexing jobs asynchronously.
- **Database Layer** — PostgreSQL (system of record + vectors), OpenSearch (search index), MinIO/S3 (large/raw object storage).
- **Visualization Layer** — Chart and graph components consuming Business Logic Layer APIs; conceptually part of the Client Layer's React app but called out separately because it has distinct library dependencies (Recharts, Cytoscape.js).
- **Infrastructure Layer** — Docker/Compose, CI/CD, and monitoring — cross-cutting, manages/observes every other layer without being a strict "client" of any of them.

7. Microservice Breakdown

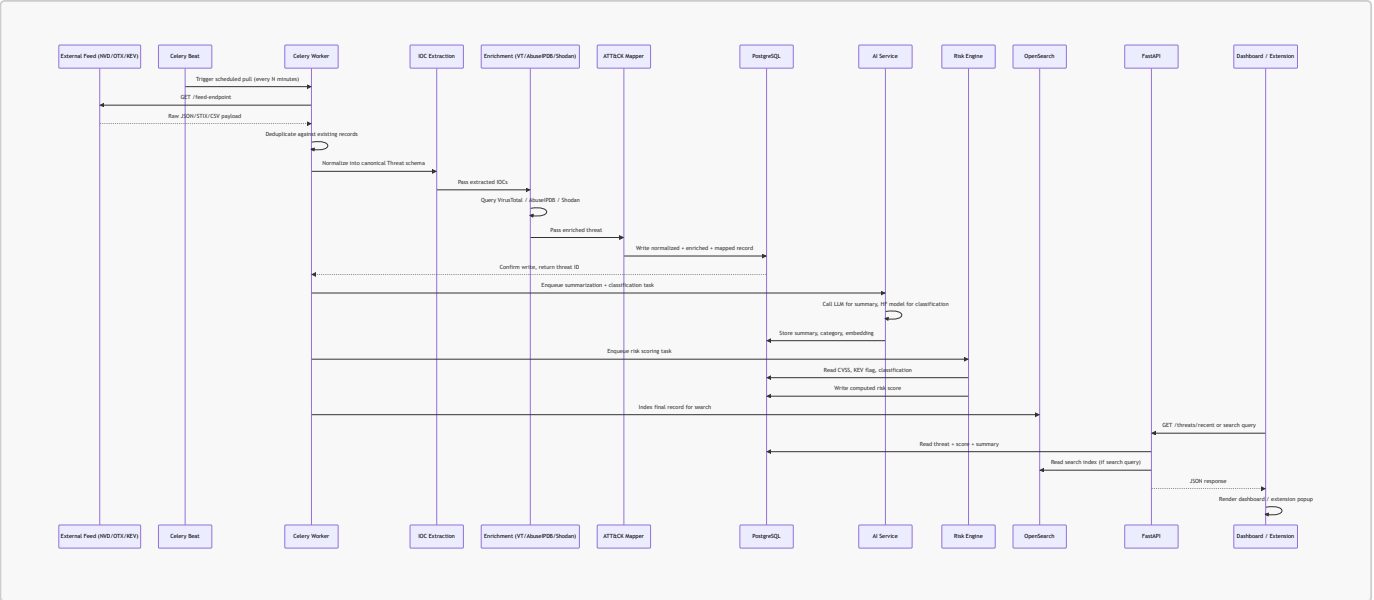
This project does not need a textbook microservices deployment (separate repos, separate d scale — that would actively work against the team's "ease of development" goal. Instead, ser **independently deployable modules within one FastAPI application + a set of Celery wo** separate deployments later (the deployment unit is what changes; the code boundary is alre

Service	Responsibilities	
Threat Feed Service	Feed connector registry, scheduling config, ingestion health trackin deduplication logic	
AI Service	Summarization, classification, embeddings, chatbot orchestration; e internal functions called by Celery tasks (not a public-facing API)	
User Service	Registration, login, JWT issuance/refresh, RBAC, profile/preferences	
Authentication Service	JWT validation middleware, OAuth flow handling — logically separ User Service even though it sits in the same codebase, since auth is by every other service	
Analytics Service	Dashboard aggregate queries (severity distribution, trends over tim for the admin panel	
Search Service	Wraps OpenSearch queries, handles structured + full-text + fuzzy I applies RBAC-aware filtering	
Risk Engine	Combines CVSS, KEV status, and AI classification/keyword-relevanc into the final 0–100 risk score	
Dashboard Service	(Frontend-facing) aggregates calls to Threat/Analytics/Search servic views the dashboard needs; mostly a thin composition layer	
Notification Service	Watchlist matching, email/webhook delivery	
Detection Mapping Service	MITRE ATT&CK technique mapping, Sigma/YARA rule linkage, IOC export (STIX/CSV)	Shreyas

Database access pattern: every service above talks to PostgreSQL through the same SQLAlchemy models (one schema, shared across services within the monolith-with-clear-boundaries approach), and to OpenSearch/Redis through their own thin client wrappers. This avoids the classic microservices trap of duplicate, drifting data models across services — a real risk for a first-time team — while keeping the *logical* boundaries clean enough to extract into true separate deployments later if the project continues past graduation.

8. Data Flow

Step-by-step path from an external feed to something visible on a user's screen.



Narrative summary:

1. **Threat Feed** — Celery Beat triggers a scheduled pull from each enabled connector (NVD, CISA KEV, AlienVault OTX, etc.).
2. **ETL / Deduplication** — The worker checks the incoming record against existing data (by CVE ID, IOC hash, etc.) to avoid duplicate processing.
3. **Cleaning / Normalization** — Source-specific formats (STIX bundles, plain JSON, CSV) are mapped into the single canonical `Threat` schema (Section 9).
4. **IOC Extraction & Enrichment** — Free-text fields are scanned for IPs/hashes/domains/URLs; each extracted IOC is enriched via VirusTotal/AbuseIPDB/Shodan reputation lookups.
5. **ATT&CK Mapping** — Where technique references exist (explicit or inferable from the advisory text), the threat is linked to MITRE ATT&CK techniques.
6. **Database Write** — The fully normalized, enriched record is written to PostgreSQL as the system of record.
7. **AI Processing** — Asynchronously, the AI Service generates a summary, a category classification, and (for RAG) an embedding — all stored back against the same threat record.
8. **Risk Engine** — Combines CVSS base score, KEV "actively exploited" flag, and AI-classification/keyword-relevance signals into a single 0–100 risk score.
9. **Search Indexing** — The finalized record is pushed into OpenSearch so it's immediately searchable.
10. **Dashboard / Extension** — The Client Layer calls the API, which reads from PostgreSQL (for detail views) and/or OpenSearch (for search/filter views), and renders the result — typically within the 5-minute SLA defined in Section 1.2, since steps 1–9 run end-to-end as one asynchronous pipeline per ingested record.

9. Database Design

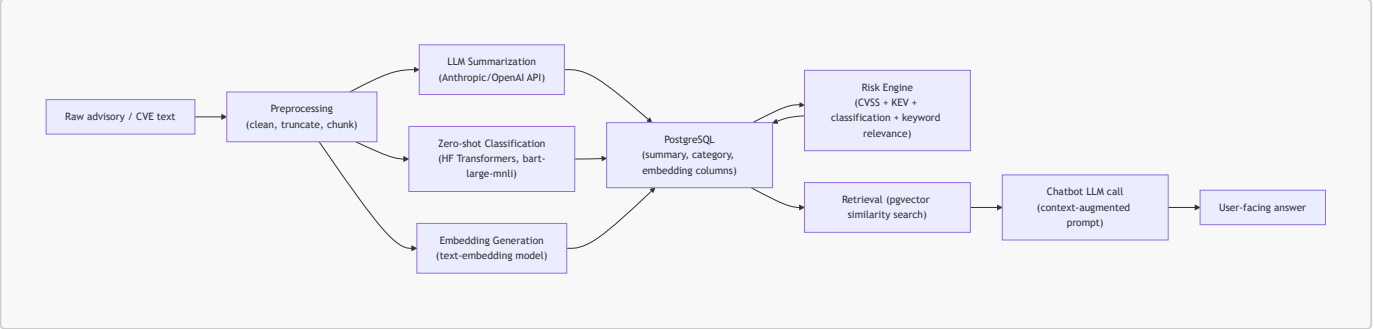
Store	What It Stores	Why This Store
PostgreSQL	System of record: `threats`, `iocs`, `threat_actors`, `campaigns`, `users`, `feed_sources`, `risk_scores`, `attack_mappings`, audit/history tables. Also stores vector embeddings via the `pgvector` extension for RAG.	Relational integrity (foreign keys between threats/IOCs/actors/campaigns matters a lot here), transactional writes, JSONB columns for the parts of a threat record that genuinely vary by source (raw source payload kept as JSONB for traceability)
OpenSearch	A denormalized, search-optimized copy of `threats` + `iocs` (full text of summaries/advisories, IOC values, tags, severity, dates)	Full-text relevance ranking and fast fuzzy/structured filtering across potentially hundreds of thousands of records — the kind of query pattern Postgres alone handles less gracefully at this volume

Store	What It Stores	Why This Store
Redis	Hot cache (recent dashboard aggregates, rate-limit counters), Celery broker + result backend, short-lived session/token blocklist	Sub-millisecond access for data that's read far more often than it changes
MinIO / S3	Raw advisory documents/PDFs, exported STIX 2.1 bundles and CSV IOC exports (FR-43), large attachments	Keeps large/binary content out of the relational DB; same API whether self-hosted (MinIO) or cloud (S3)
pgvector (within PostgreSQL)	Embedding vectors for each threat's summary text, used by the optional RAG chatbot to retrieve relevant context	Co-locating vectors with the relational data they describe avoids a consistency problem between two separate databases

PostgreSQL is the single source of truth. OpenSearch is always a derived, rebuildable copy — if it's ever lost or corrupted, it can be fully reindexed from PostgreSQL. This rule (one source of truth, everything else derived) is what keeps the multi-database setup from becoming a data-consistency nightmare for a first-time team.

---

10. AI Architecture



LLM — Used specifically for tasks that genuinely need generative, language-fluent output: summarization (FR-12) and the optional chatbot's final answer generation (FR-15). Calls go through a thin internal wrapper (not scattered raw SDK calls) so the model/provider can be swapped and every call can be logged per FR-16.

Embeddings — Generated once per threat record (on ingestion) using a standard sentence-embedding model, stored via `pgvector`. Re-used for every future similarity search rather than recomputed per query.

Threat Summarization — Prompted as a constrained, structured task: "summarize this CVE/advisory in 2–4 plain-English sentences for a security analyst, focusing on what's affected, how it's exploited, and what to do." Output length and tone are constrained by the prompt, not left open-ended, to keep summaries dashboard-sized.

Risk Prediction — Deliberately a *hybrid* model, not a pure black-box ML score: CVSS base score (deterministic) + KEV "actively exploited" boolean (deterministic) + AI-derived category and a keyword-relevance signal (e.g., does this threat mention software the org/demo dataset actually runs) are combined via a small, explainable scikit-learn model (e.g., gradient boosting or even a transparent weighted formula for v1). This matters for a security tool specifically: an analyst needs to be able to see *why* something scored 87/100, not just trust an opaque number.

Classification — Zero-shot classification via Hugging Face Transformers against a fixed label set (RCE, Privilege Escalation, DoS, Info Disclosure, etc.) — chosen specifically because the team has no labeled training set yet; zero-shot needs none.

Prompt Engineering — Prompts are version-controlled (in a `prompts/` directory, not buried inline in code) so changes can be reviewed and A/B compared. Each prompt template specifies: role/persona, output format constraints, and a small worked example for consistency.

Model Deployment — LLM calls hit external provider APIs (no self-hosting of the generative model — not worth the infra burden for a student project). The Hugging Face classification model and the embedding model run locally inside the AI service container (CPU-fine for the model sizes involved here).

Inference Pipeline — All AI calls happen inside Celery tasks, never inline in an API request/response cycle, so a slow or rate-limited external call never blocks a user-facing dashboard request (ties directly to the Availability NFR: AI failure degrades gracefully — the threat record simply shows without a summary until the retry succeeds).

Fine-tuning (optional/future) — Once real usage produces enough labeled examples (analysts correcting AI categories, confirming/rejecting risk scores), a fine-tuned classifier becomes viable. Documented as a v2 path, not a v1 requirement, since it needs data the project doesn't have on day one.

---

11. Threat Intelligence Pipeline

Stage	What Happens	Primary Tooling
Data Sources	NVD/CVE API, CISA KEV, AlienVault OTX, VirusTotal, AbuseIPDB, Shodan, (optional) MISP/OpenCTI	Section 12 details each
Feed Collection	Scheduled polling per source, with source-specific connector classes implementing a shared interface (`fetch()`, `parse()`)	Celery Beat + Celery Workers
Normalization	Source-specific payloads mapped into the canonical `Threat` Pydantic/SQLAlchemy schema	Python dataclasses/Pydantic models, one mapper per source
Deduplication	Match on natural keys (CVE ID, IOC value + type) before writing; updates to existing records create a new version row rather than overwriting silently	PostgreSQL upsert logic + history table
IOC Extraction	Regex + lightweight NLP (e.g., `re` patterns for IPs/hashes/domains, spaCy or simple heuristics for filenames/registry keys) pulled from free-text advisory bodies	Python `re`, optionally spaCy
Enrichment	Each extracted IOC is checked against VirusTotal/AbuseIPDB/Shodan for reputation/context	Async HTTP clients (`httpx`), with response caching in Redis to respect free-tier rate limits
Storage	Canonical record → PostgreSQL; raw source payload retained as JSONB for traceability/debugging	PostgreSQL
AI Processing	Summarization, classification, embedding (Section 10)	AI Service (Celery tasks)
Analytics	Aggregate queries for dashboard charts (severity trends, top affected vendors, campaign timelines)	Analytics Service (SQL aggregates + cached results)
Visualization	Rendered as dashboard charts, IOC search results, and actor/campaign graphs	React + Recharts + Cytoscape.js

---

12. External Integrations

Integration	Purpose	Integration Method	Notes
NVD API	Primary source of CVE records (CVSS scores, descriptions, references)	REST API polling, paginated, with an API key for higher rate limits	Free; rate-limited without a key, much higher limit with one — register early
CVE API (MITRE CVE.org)	Authoritative CVE ID records, sometimes ahead of full NVD enrichment	REST API polling	Useful as a secondary source for CVE IDs not yet fully scored in NVD

Integration	Purpose	Integration Method	Notes
MITRE ATT&CK	Technique/tactic reference data for the ATT&CK Mapper	Download the ATT&CK STIX bundle periodically (it's a dataset, not a live feed) and load into PostgreSQL	Static-ish reference data — refresh monthly, not real-time
VirusTotal	Reputation/enrichment for file hashes, URLs, domains, IPs	REST API, per-IOC lookup	Free tier is rate-limited (4 req/min on the public API) — must cache aggressively and queue lookups rather than calling synchronously
AlienVault OTX	Community-sourced IOC pulses and malware intelligence	REST API (OTX DirectConnect SDK or raw REST)	Good source of fresh, community-tagged IOCs; quality varies by contributor, so enrichment/cross-referencing matters
AbuseIPDB	IP reputation/abuse-confidence scoring	REST API, per-IP lookup	Free tier rate-limited; cache results, especially for IPs seen across many threats
Shodan	Internet-facing asset/exposure context for IPs (open ports, banners, known vulnerable services)	REST API, per-IP lookup	Paid API beyond a small free quota — used selectively (e.g., only for high-risk IOCs) to control cost
CISA KEV	Authoritative "actively exploited in the wild" flag for CVEs	Periodic download of the published JSON catalog (it's a flat file, not a query API)	Directly powers the "actively exploited" boolean in the Risk Engine — high-value, low-effort integration
MISP (optional)	Structured threat-sharing platform — could both consume from and contribute to community MISP instances	MISP REST API / PyMISP library	Optional v2 stretch goal; valuable for STIX/IOC interoperability but adds setup overhead (needs its own MISP instance or community access)
OpenCTI (optional)	Structured CTI knowledge graph platform, strong for actor/campaign/relationship modeling	OpenCTI GraphQL API	Optional; could eventually replace some of the custom actor/campaign modeling in Section 9, but introduces a second graph-style data model to reconcile with PostgreSQL — deferred past v1

13. Security Architecture

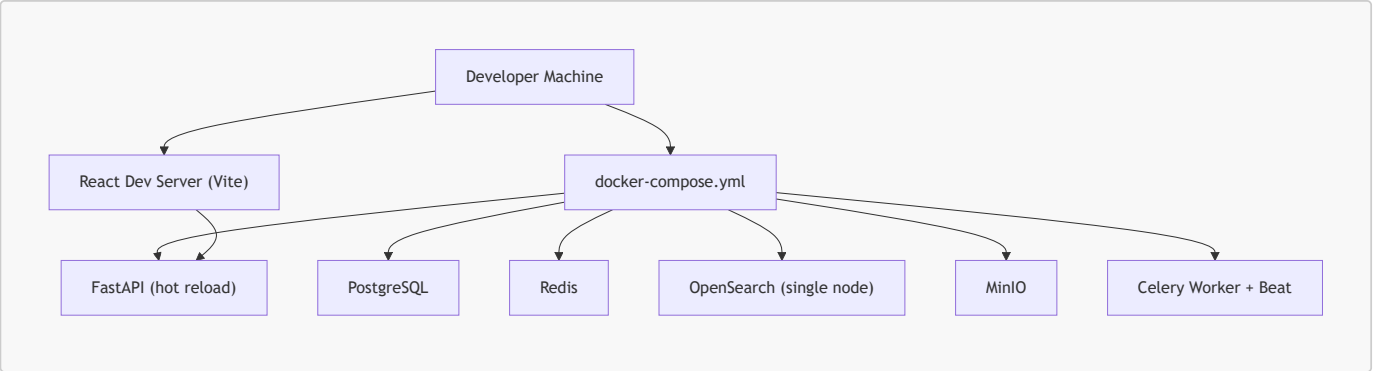
A platform whose entire purpose is cybersecurity intelligence has to hold itself to a higher security bar than a typical student project — this is one of the more important credibility signals in the whole build.

Control	Implementation
JWT Authentication	Short-lived access tokens (15 min) + longer-lived refresh tokens (7 days), signed with a strong secret (rotated via environment/secret manager, never hardcoded)
OAuth	Google/GitHub OAuth via Authlib as an alternative login path; OAuth identities linked to the same `users` table, not a separate auth system
RBAC	Three roles minimum (Admin, Analyst, Viewer) enforced via a FastAPI dependency checked on every protected route — never trust a role claim without re-validating it server-side per request

Control	Implementation
HTTPS	Enforced everywhere in any non-local environment; HTTP requests redirected to HTTPS at the reverse proxy (NGINX/Caddy) layer
Encryption	TLS in transit everywhere; sensitive fields (e.g., API keys for external feeds, stored in `feed_sources` config) encrypted at rest using a library like `cryptography`'s Fernet, with the encryption key itself held in the secrets manager, not the database
Secrets Management	Environment variables for local dev (`.env`, git-ignored); a real secrets manager (Docker secrets, or a cloud provider's secret manager) in any deployed environment — no secrets ever committed to the repo, enforced via a pre-commit hook and CI secret-scanning
API Security	Every endpoint validated via Pydantic schemas (rejects malformed input before it reaches business logic); consistent error responses that don't leak stack traces or internal details in non-debug mode
Rate Limiting	Per-user and per-IP rate limiting at the API gateway layer (e.g., `slowapi` for FastAPI, backed by Redis) — protects both the platform itself and the external feed APIs it depends on (which have their own rate limits to respect)
Input Validation	Pydantic models on every request body; additional sanitization on any field that gets rendered back into the UI (defense in depth alongside frontend escaping)
SQL Injection Protection	SQLAlchemy ORM with parameterized queries exclusively — no raw string-interpolated SQL anywhere in the codebase, enforced via code review and a linter rule where possible
XSS Protection	React's default JSX escaping handles most of this; any place that must render raw HTML (e.g., a formatted advisory) goes through a sanitization library (e.g., DOMPurify) before rendering, never `dangerouslySetInnerHTML` on unsanitized content
CSRF Protection	Since auth is JWT-in-header (not cookie-based session auth) for the API, classic CSRF risk is significantly reduced; if cookies are used for refresh tokens, `SameSite=Strict` + CSRF tokens are added for those specific endpoints
Audit Logs	Every write to `threats`, `users`, and `feed_sources` tables is recorded in an append-only audit table (who, what, when, before/after) — critical for a platform that itself analyzes security incidents to be auditable
Extension-Specific Security	The Chrome extension never embeds long-lived credentials in its bundled code; the JWT is stored via `chrome.storage.local` (not `localStorage`, which content scripts on other sites could access), and the extension's permissions in `manifest.json` are scoped to the minimum needed (active tab + the Hub's own API origin)
Dependency Security	`pip-audit` / `npm audit` run in CI on every push; flagged vulnerable dependencies block merges above a defined severity threshold

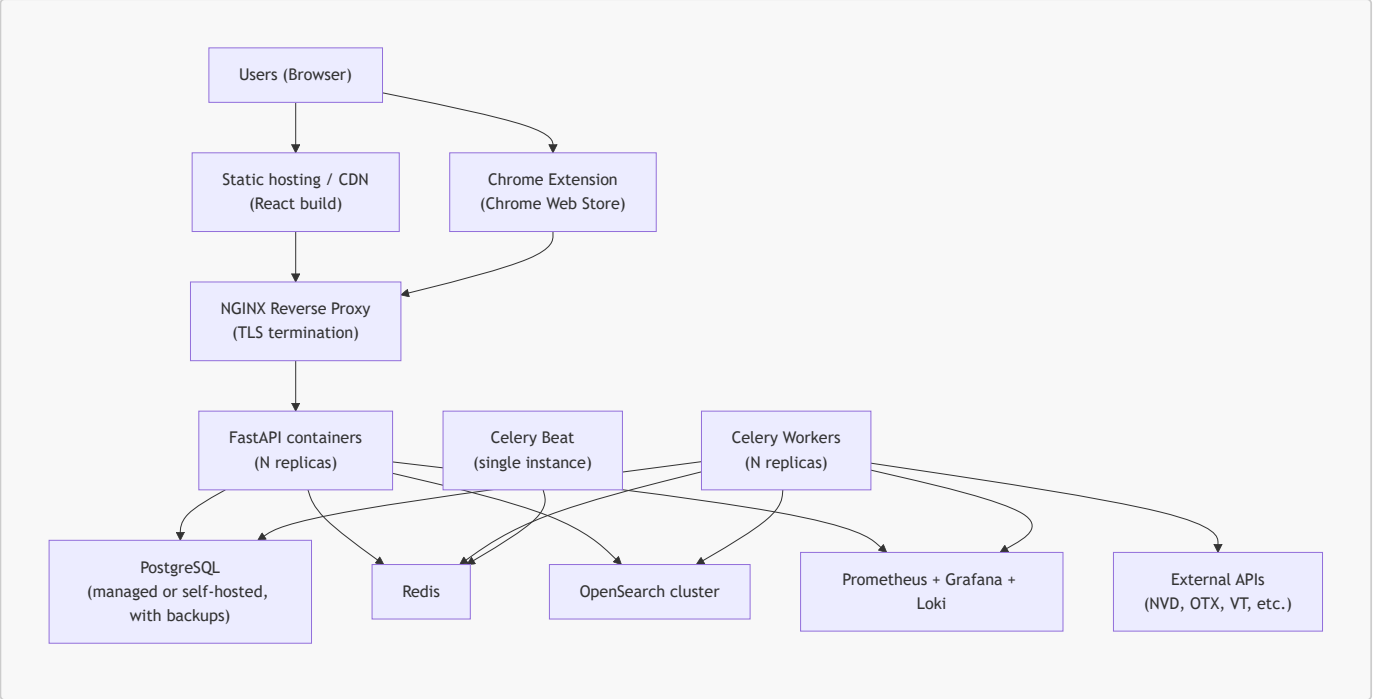
14. Deployment Architecture

14.1 Development Architecture



All services run via a single `docker-compose.yml` for local development, with the React frontend run separately via Vite's dev server (for hot module reload) and proxying API calls to the FastAPI container. This means any team member can clone the repo, run `docker compose up` plus `npm run dev`, and have the entire stack running locally within minutes.

14.2 Production Architecture



- **Docker** — Every service ships as its own image, built and pushed by CI.
- **Docker Compose** — Sufficient for the actual production deployment at this project's scale (single VM/VPS or one small cloud instance running all containers).
- **Kubernetes (optional)** — Documented upgrade path if the project needs to run across multiple nodes (e.g., genuinely high ingestion volume or many concurrent users): the same Docker images would be deployed via Kubernetes Deployments/Services instead of Compose, with Celery workers as a separately-scaled Deployment from the API.
- **NGINX** — TLS termination, reverse proxy to the FastAPI containers, and serving the built React static assets.
- **GitHub Actions** — On merge to `main`: run tests → build images → push to a container registry (GitHub Container Registry) → (optionally) SSH-deploy / pull-and-restart on the target VM.
- **Cloud Deployment** — A single VPS or a small cloud VM (2–4 vCPU, 8GB RAM is a reasonable starting point given OpenSearch's memory appetite) running the full Docker Compose stack is sufficient for the demo and even light real usage; **Celery Beat must run as exactly one instance** (never scaled to N) to avoid duplicate scheduled jobs.

---

15. Folder Structure

```
cti-hub/
├── backend/
│   ├── app/
│   │   ├── api/
│   │   │   ├── v1/
│   │   │   │   ├── routers/
│   │   │   │   │   ├── auth.py
│   │   │   │   │   ├── threats.py
│   │   │   │   │   ├── feeds.py
│   │   │   │   │   ├── search.py
│   │   │   │   │   ├── users.py
│   │   │   │   │   ├── analytics.py
│   │   │   │   │   ├── ai.py
│   │   │   │   │   └── admin.py
│   │   │   │   └── deps.py          # shared dependencies (auth, db session)
│   │   ├── core/
│   │   │   ├── config.py            # settings via pydantic-settings
│   │   │   ├── security.py          # JWT, password hashing
│   │   │   └── logging.py
│   │   ├── models/                  # SQLAlchemy ORM models
│   │   │   ├── threat.py
│   │   │   ├── ioc.py
│   │   │   ├── user.py
│   │   │   ├── actor.py
│   │   │   └── feed_source.py
│   │   ├── schemas/                 # Pydantic request/response schemas
│   │   ├── services/                # business logic layer
│   │   │   ├── threat_service.py
│   │   │   ├── risk_engine.py
│   │   │   ├── search_service.py
│   │   │   └── notification_service.py
│   │   ├── ai/
│   │   │   ├── summarizer.py
│   │   │   ├── classifier.py
│   │   │   ├── embeddings.py
│   │   │   ├── chatbot.py
│   │   │   └── prompts/
│   │   ├── connectors/              # feed connector plugins
│   │   │   ├── base.py              # shared connector interface
│   │   │   ├── nvd.py
│   │   │   ├── cisa_kev.py
│   │   │   ├── otx.py
│   │   │   ├── virustotal.py
│   │   │   ├── abuseipdb.py
│   │   │   └── shodan.py
│   │   ├── pipeline/
│   │   │   ├── normalize.py
│   │   │   ├── dedupe.py
│   │   │   ├── ioc_extraction.py
│   │   │   ├── enrichment.py
│   │   │   └── attack_mapper.py
│   │   ├── tasks/                   # Celery task definitions
│   │   │   ├── celery_app.py
│   │   │   ├── ingestion_tasks.py
│   │   │   ├── ai_tasks.py
│   │   │   └── indexing_tasks.py
│   │   ├── db/
│   │   │   ├── session.py
│   │   │   └── migrations/          # Alembic
│   │   └── main.py                  # FastAPI app entrypoint
│   ├── tests/
```

```
│   ├── pyproject.toml
│   └── Dockerfile
├── frontend/
│   ├── src/
│   │   ├── components/             # shared with extension where possible
│   │   ├── pages/
│   │   │   ├── Dashboard.tsx
│   │   │   ├── ThreatDetail.tsx
│   │   │   ├── Search.tsx
│   │   │   ├── Actors.tsx
│   │   │   └── Admin.tsx
│   │   ├── api/                    # typed API client (generated from OpenAPI)
│   │   ├── hooks/
│   │   ├── store/                  # auth/session state
│   │   └── App.tsx
│   ├── package.json
│   └── Dockerfile
├── extension/
│   ├── src/
│   │   ├── background.ts           # MV3 service worker
│   │   ├── content-script.ts       # selection lookup
│   │   ├── popup/                  # React popup, reuses frontend/src/components
│   │   └── storage.ts              # chrome.storage wrapper
│   ├── manifest.json
│   └── package.json
├── infra/
│   ├── docker-compose.yml
│   ├── docker-compose.dev.yml
│   ├── nginx/
│   └── monitoring/
│       ├── prometheus.yml
│       └── grafana/
├── docs/                           # MkDocs site (this document lives here)
├── .github/
│   └── workflows/
│       ├── backend-ci.yml
│       ├── frontend-ci.yml
│       └── extension-ci.yml
└── README.md
```

16. API Design

All endpoints are prefixed `/api/v1`. Authentication via `Authorization: Bearer <JWT>` unless noted.

Authentication

Method	Endpoint	Description
POST	`/auth/register`	Create a new user account
POST	`/auth/login`	Authenticate, returns access + refresh tokens
POST	`/auth/refresh`	Exchange refresh token for new access token
POST	`/auth/logout`	Invalidate refresh token (blocklist in Redis)
GET	`/auth/oauth/{provider}`	Initiate OAuth flow (Google/GitHub)
GET	`/auth/oauth/{provider}/callback`	OAuth callback handler

Threats

Method	Endpoint	Description
GET	`/threats`	List/paginate threats (filters: severity, date range, vendor, category)
GET	`/threats/{id}`	Full detail: summary, raw advisory, IOCs, risk breakdown, ATT&CK mapping
GET	`/threats/recent`	Recent critical threats for dashboard overview
GET	`/threats/{id}/related`	Related threats (same actor/campaign/IOC overlap)
POST	`/threats/{id}/reprocess`	(Admin) re-run AI summarization/classification for one record

Feeds

Method	Endpoint	Description
GET	`/feeds`	List configured feed sources and health status
POST	`/feeds`	(Admin) Add a new feed source
PATCH	`/feeds/{id}`	(Admin) Enable/disable, adjust polling frequency
POST	`/feeds/{id}/trigger`	(Admin) Manually trigger an immediate pull

Indicators of Compromise (IOCs)

Method	Endpoint	Description
GET	`/iocs/search`	Search IOCs by value/type (exact + fuzzy)
GET	`/iocs/{id}`	IOC detail with enrichment data and linked threats
GET	`/iocs/export`	Export IOC set as CSV or STIX 2.1 bundle

Threat Actors & Campaigns

Method	Endpoint	Description
GET	`/actors`	List threat actors
GET	`/actors/{id}`	Actor detail: TTPs, associated campaigns/IOCs, ATT&CK techniques
GET	`/campaigns`	List campaigns
GET	`/campaigns/{id}`	Campaign detail with timeline and related IOCs/threats

Search

Method	Endpoint	Description
GET	`/search`	Unified full-text search across threats/advisories
GET	`/search/suggest`	Autocomplete suggestions

AI

Method	Endpoint	Description
POST	`/ai/chat`	Chatbot query (optional feature), RAG-augmented
GET	`/ai/usage`	(Admin) inference logs, cost/latency metrics per FR-16

Risk Scores

Method	Endpoint	Description
GET	`/threats/{id}/risk`	Risk score breakdown (CVSS contribution, KEV flag, classification weight)

Users & Dashboard

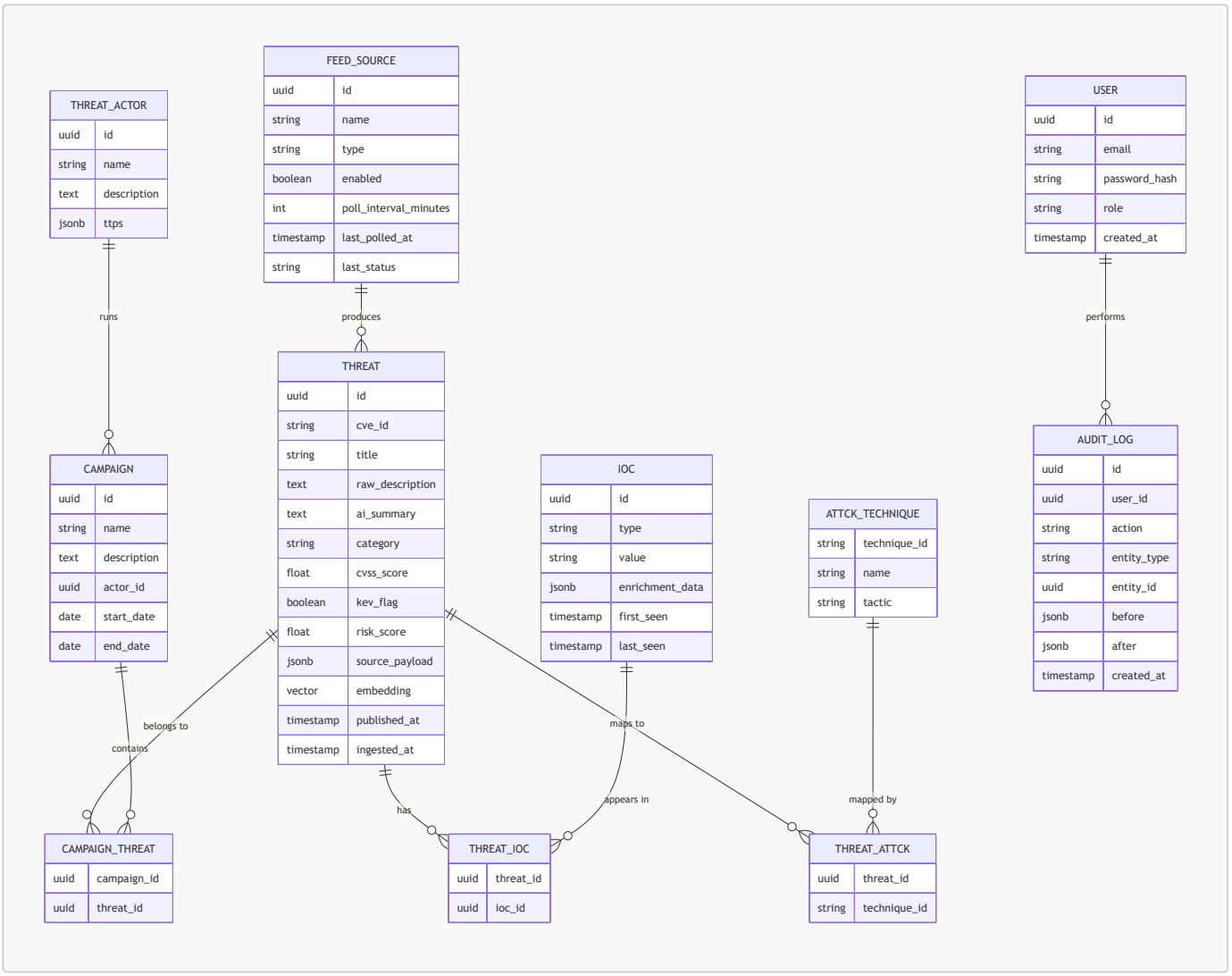
Method	Endpoint	Description
GET	`/users/me`	Current user profile + preferences
PATCH	`/users/me`	Update preferences (watchlists, default filters)
GET	`/dashboard/overview`	Aggregated dashboard payload (recent critical, trends, top IOCs)
GET	`/analytics/severity-trend`	Time-series severity distribution data for charts

Admin

Method	Endpoint	Description
GET	`/admin/users`	List/manage users and roles
GET	`/admin/metrics`	System health metrics (ingestion volume, error rates, AI cost)

---

17. Database ER Diagram



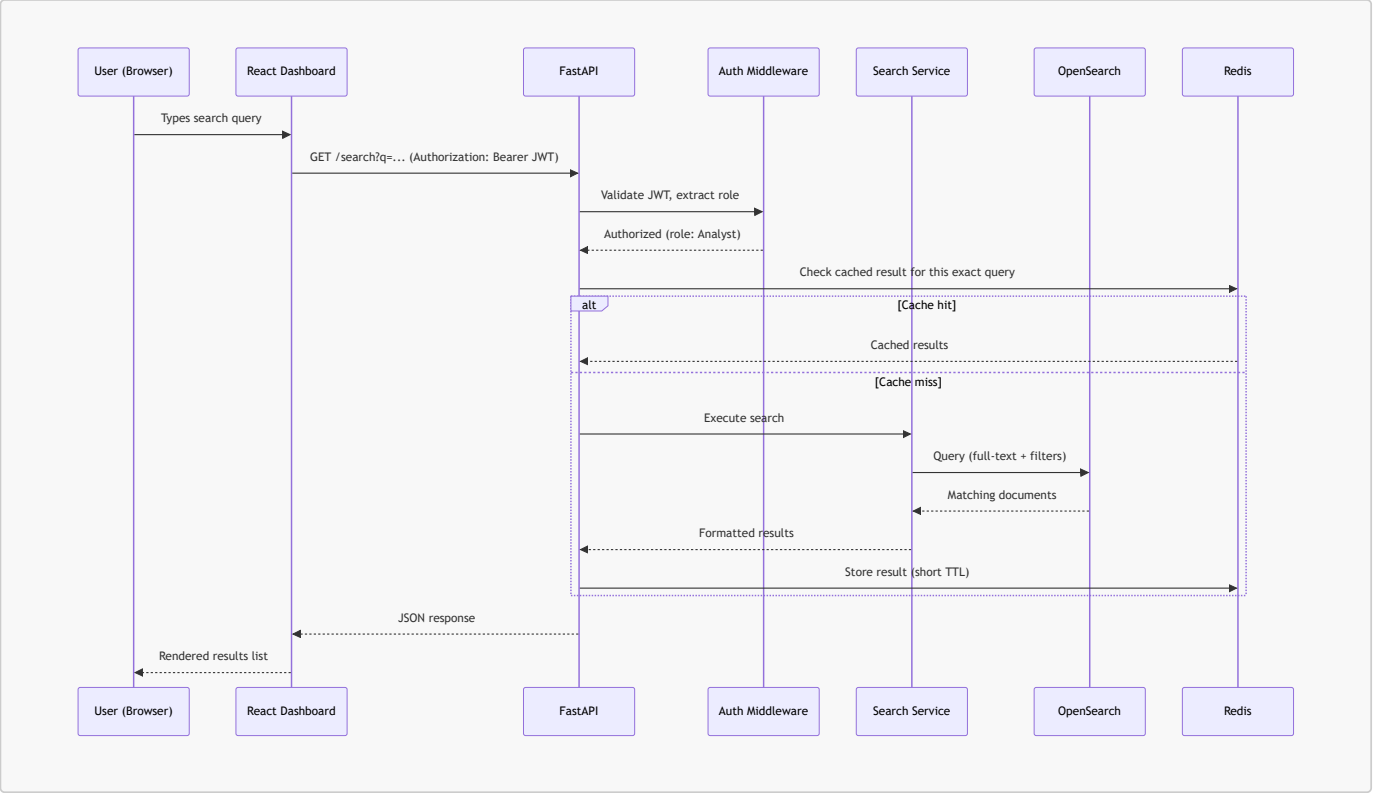
Key relationships:

- A `Threat` can have many `IOCs` and vice versa (many-to-many via `THREAT_IOC`).
- A `Campaign` belongs to one `ThreatActor` but spans many `Threats` (many-to-many via `CAMPAIGN_THREAT`, since a threat could plausibly relate to more than one tracked campaign).
- A `Threat` maps to zero or more `ATTCK_TECHNIQUE` rows.
- Every `Threat` traces back to the `FeedSource` it was ingested from.
- Every meaningful write by a `User` is recorded in `AUDIT_LOG`.

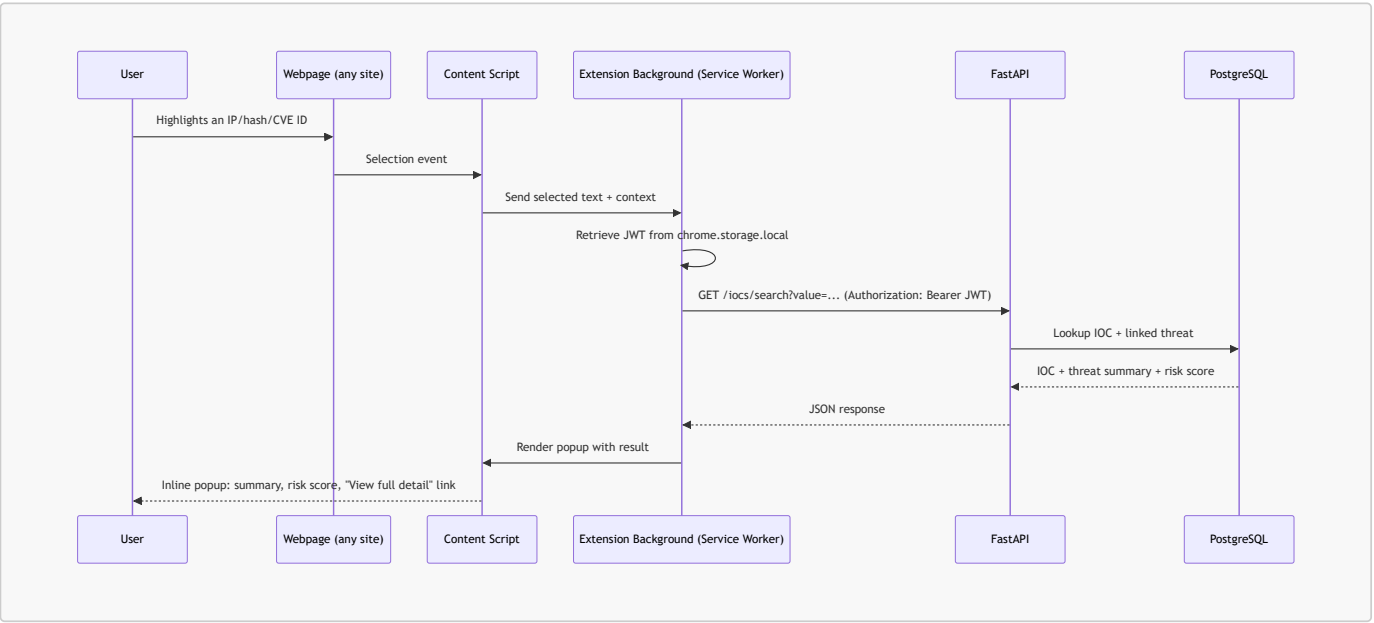
---

18. Sequence Diagrams

18.1 User Searches for a Threat (Web Dashboard)



18.2 Chrome Extension IOC Lookup



18.3 Scheduled Ingestion → AI Processing (Background)

Covered in full in Section 8's data-flow sequence diagram; that diagram is the canonical reference for this flow and is not duplicated here to avoid drift between two copies of the same sequence.

19. Compatibility & Integration Report

This section addresses cross-cutting integration questions directly, including where real friction can occur and how to mitigate it.

Frontend ↔ Backend. React (TypeScript) talks to FastAPI exclusively over JSON REST, with the API contract generated automatically from FastAPI's OpenAPI schema — the frontend can generate a typed client (`openapi-typescript-codegen` or similar) directly from that schema, so the two never drift silently out of sync. CORS must explicitly allow both the web app's origin and the Chrome extension's `chrome-extension://<id>` origin.

Backend ↔ AI. Because the AI layer is plain Python (Anthropic/OpenAI SDKs, Hugging Face Transformers, scikit-learn) running in the same language and largely the same containers as the rest of the backend, there's no serialization boundary or RPC layer needed between "backend" and "AI" the way there would be if AI were a separate-language service. The only real boundary is that AI calls run inside Celery tasks, not inline in FastAPI request handlers — this is a deliberate architectural choice (Section 10), not a technical limitation.

Backend ↔ Databases. SQLAlchemy (async) is the single ORM layer FastAPI uses for PostgreSQL; `opensearch-py` and `redis-py` are used directly (no ORM abstraction needed for those, since they're accessed via query/cache patterns rather than relational modeling). Alembic manages all PostgreSQL schema migrations so schema changes are versioned and reviewable rather than applied ad hoc.

AI ↔ Vector DB. `pgvector` lives inside the same PostgreSQL instance as the relational threat data, so embedding writes and threat-record writes can happen in the same transaction — there's no two-database consistency problem to manage, which is the main integration risk a standalone vector DB (Qdrant/Weaviate) would have introduced.

AI ↔ LLM APIs. Anthropic/OpenAI SDKs are async-HTTP-based and run fine inside Celery's worker processes; the main integration risk is rate limiting and cost, mitigated by caching unchanged content (don't re-summarize a CVE that hasn't been updated) and by setting hard per-task timeouts so a slow provider response can't stall a worker indefinitely.

Elasticsearch ↔ PostgreSQL *(read as OpenSearch ↔ PostgreSQL, per Section 4.4's selection)*. These are two independent systems linked by an application-level dual-write: PostgreSQL writes happen first (source of truth), then a Celery task indexes the same record into OpenSearch. The integration risk is index drift if that indexing task fails silently — mitigated by alerting (Prometheus) on indexing task failures and by providing an admin "reindex all" operation that rebuilds OpenSearch entirely from PostgreSQL on demand.

Redis ↔ Backend. Redis serves three distinct roles (cache, Celery broker, rate-limit counter store) from one running instance. The integration risk is key-namespace collisions between these roles — mitigated by using separate Redis logical database numbers or consistent key prefixes (`cache:`, `celery:`, `ratelimit:`) per role.

Docker ↔ Kubernetes. Since every service is already containerized for Docker Compose, migrating to Kubernetes later (if the project scales past v1) is a deployment-layer change, not a code change — the same images are simply scheduled by Kubernetes Deployments/Services instead of Compose. The one component requiring care is Celery Beat, which must run as a singleton regardless of orchestrator (a Kubernetes Deployment with `replicas: 1`, not a scaled Deployment).

CI/CD compatibility. GitHub Actions builds and tests both the Python backend (pytest) and the TypeScript frontend/extension (Vitest) in parallel matrix jobs, then builds and pushes Docker images tagged by commit SHA. Because every runtime dependency is pinned (Python via `pyproject.toml`/lockfile, JS via `package-lock.json`, Docker images via explicit version tags), CI builds are reproducible rather than depending on whatever "latest" happened to resolve to on a given day — this is the single most common integration failure mode for student teams and is explicitly designed against here.

Cloud compatibility. Nothing in this stack requires a specific cloud provider: PostgreSQL, Redis, OpenSearch, and MinIO all have official Docker images and can run identically on a bare VPS, any major cloud's VM offering, or a teammate's laptop for local development. The only place a specific cloud's managed service could be substituted (e.g., AWS RDS instead of self-hosted PostgreSQL, or real S3 instead of MinIO) is a drop-in replacement at the infrastructure layer with no application code changes, because the application talks to the *API* (PostgreSQL wire protocol, S3 API) not to a provider-specific SDK.

Version conflicts to watch:

- `pgvector` requires using the `pgvector/pgvector` Postgres Docker image (or manually installing the extension) — the default `postgres` image does not include it.
- Hugging Face Transformers and PyTorch/TensorFlow versions must be pinned together deliberately; an unpinned `transformers` upgrade can silently require a newer `torch` than what's installed in the container.
- OpenSearch client library major versions must match the OpenSearch server's major version — `opensearch-py` releases track this and should be pinned accordingly in `pyproject.toml`.
- Chrome Extension Manifest V3's service-worker lifecycle differs meaningfully from the older Manifest V2 persistent background page model assumed by a lot of older tutorials/Stack Overflow answers — the team should reference current MV3 documentation specifically, not generic "Chrome extension" guides that may predate MV3.

---

20. Final Recommended Stack

Layer	Final Choice	One-Line Justification
Frontend (web)	**React + TypeScript + Vite**	Richest ecosystem for dashboard UI; shares components with the extension popup
Browser Extension	**Chrome Extension, Manifest V3, React popup**	Current required standard; reuses web app's components
Backend / API	**FastAPI (Python)**	Async, AI-friendly, keeps backend/data/AI in one language for a small team
Primary Database	**PostgreSQL (+ pgvector)**	Strong relational integrity + native vector support, no second DB needed for RAG
Search Engine	**OpenSearch**	Open-source-licensed, Elasticsearch-equivalent capability for IOC/full-text search
Cache / Broker	**Redis**	One component serving cache, Celery broker, and rate-limiting needs
Task Queue	**Celery (+ Celery Beat)**	Native Python, maps directly onto scheduled feed polling and async AI tasks
AI / LLM Access	**Direct Anthropic/OpenAI SDK calls, narrow LangChain/LlamaIndex use for RAG only**	Full visibility into prompts/costs (needed for FR-16), avoids unnecessary abstraction
ML / Classification	**scikit-learn (risk scoring) + Hugging Face Transformers (zero-shot classification)**	Matches the data the team actually has (no labeled training set yet)
Vector Store	**pgvector**	Co-located with relational data, one less service to operate
Authentication	**Custom JWT (FastAPI) + OAuth via Authlib**	No vendor lock-in, doubles as a genuine security-engineering deliverable
Object Storage	**MinIO (self-hosted, S3-compatible)**	Free, portable, identical API to AWS S3 if the team migrates later
Containerization	**Docker + Docker Compose**	Matches the team's actual deployment scale; Kubernetes is a documented future option, not a v1 requirement
CI/CD	**GitHub Actions**	Free, integrates directly with the team's existing GitHub repo

Layer	Final Choice	One-Line Justification
Monitoring	**Prometheus + Grafana + Loki**	Open-source, Docker-Compose-friendly, industry-standard observability stack
Documentation	**MkDocs Material + FastAPI auto-generated OpenAPI docs**	Markdown-native, stays in sync with code automatically
Visualization	**Recharts + Cytoscape.js**	Recharts for standard dashboards, Cytoscape.js for actor/campaign relationship graphs
Testing	**pytest + Vitest/React Testing Library**	Native to each layer's language, both CI-friendly

Why this balances the project's stated priorities

- **Ease of development** — A 6-person team gets one primary backend language (Python) shared across API, data engineering, and AI work, and one primary frontend language (TypeScript) shared across the web app and the extension. Nobody has to context-switch across more than two languages total.
- **Industry relevance / resume value** — FastAPI, React, PostgreSQL, Redis, Celery, Docker, OpenSearch, Prometheus/Grafana, and LLM-API integration are all skills that map directly onto real job postings in both backend/data engineering and applied-AI roles — arguably more transferable than a from-scratch Kubernetes/Kafka setup that this project's actual scale doesn't justify.
- **Scalability** — Every component chosen has a documented, low-friction upgrade path (Celery→Kafka if ingestion volume grows; Docker Compose→Kubernetes if multi-node scheduling is needed; pgvector→Qdrant if RAG usage becomes heavy) without requiring an application-level rewrite.
- **AI integration** — AI is embedded at the architecture level (its own layer, its own async task path, its own logging requirement) rather than bolted on as an afterthought, while still keeping the AI code simple enough for one team member (Duradarshee) to own end-to-end.
- **Cybersecurity best practices** — JWT + RBAC + encryption-at-rest + audit logging + rate limiting + dependency scanning are all present from v1, not deferred — appropriate for a project whose entire subject matter is security.
- **Data engineering best practices** — Clear ETL pipeline stages (Section 11), a single canonical schema with full version history, and a strict "one source of truth, everything else derived" rule between PostgreSQL and OpenSearch.
- **Performance** — Async FastAPI + Redis caching + OpenSearch for search-heavy queries keeps the read paths fast without needing premature optimization elsewhere.
- **Open-source preference** — Every component in the final table (with the sole intentional exception of the LLM API itself, which has no equivalently capable open-source self-hosted alternative within this project's compute budget) is open-source and free to run.
- **Cost-effectiveness** — The entire stack runs on a single modest VPS or a student cloud credit allocation; the only recurring cost is LLM API usage, which is controlled via caching, batching, and task timeouts.